

MODUL 248 — TIE INTERNATIONAL

Challenge C — Referral System auf der Cardano Blockchain

TalentChain

Technische Dokumentation

Version	1.0
Datum	28.Mai 2026
Netzwerk	Cardano Preprod Testnet
Smart Contracts	Aiken → Plutus V3

1. Projektübersicht	3
1.1 Projektziele	3
2. Akteure	4
3. Use Cases	4
Zusammenfassung — Übersicht aller Use Cases	4
UC-01 — Wallet verbinden	5
UC-02 — App-Download teilen	5
UC-03 — Root-User durch Admin registrieren (L1 Ambassador)	5
UC-04 — Einladung annehmen (Referral registrieren)	6
UC-05 — Doppelregistrierung verhindern	6
UC-06 — Eigene Hierarchie anzeigen	7
UC-07 — Match-Event auslösen und Rewards verteilen	7
UC-08 — Rewards automatisch verteilen (Smart Contract)	8
UC-09 — Kette traversieren (Off-Chain)	8
UC-10 — Reward-Transaktion anzeigen	8
UC-11 — Contract-Laufzeit verwalten	9
UC-12 — User sperren (Blacklist)	9
4. Technologie-Stack	10
4.1 Smart Contracts — Aiken	10
4.2 Off-Chain SDK — Lucid Evolution	10
4.3 Cardano Serialization Library (CSL)	10
4.4 Blockfrost	11
4.5 Backend — Node.js + TypeScript	11
4.6 Datenbank — PostgreSQL + Prisma	11
4.7 Authentifizierung — JWT + bcrypt	11
4.8 Frontend — Next.js 14 + React	11
4.9 Zusammenfassung	12
5. Systemarchitektur	13
6. Sequenzdiagramme	13
6.1 Registrierung mit Einladungslink (3-TX-Flow)	13
6.2 Reward-Verteilung (Match-Event)	15
6.3 Identity NFT Minting — Aiken Validator Interaktion	15
7. Testkonzept	17
7.1 Warum Testing bei Smart Contracts besonders wichtig ist	17
7.2 Eingesetzte Test-Methoden	17
7.3 API-Tests (Postman) — Happy Path	17
Auth-Routes /api/auth	17
Match-Routes /api/match	18
Reward-Routes /api/rewards	18
Admin-Routes /api/admin	18
7.4 Negativtests — Fehlerfälle & Sicherheit	18
Authentifizierung & Zugriffskontrolle	19
Registrierung & Doppelregistrierung	19

Match-Events & Rewards	19
Blacklist & Referral-Kette	20
On-Chain Sicherheit	20
7.5 Manuelle End-to-End Tests	21
Registrierung & Wallet	21
Admin Dashboard	21
Referral-Kette & Reward-Verteilung	22
On-Chain Verifikation (Cardanoscan)	22
7.6 Bewusst nicht getestete Bereiche	22
8. Reflexion — Feedback aus der Zwischenpräsentation	23
Aufgenommenes Feedback	23
1. SDK Wahl: Evolution SDK zu Lucid Evolution	23
2. Off Chain berechnen, On Chain verifizieren	23
3. Kein eigener Cardano Node in der Demo Phase	23
Nicht aufgenommenes oder bewusst zurückgestelltes Feedback	23
1. Dual Signing durch Einlader und neue Person	23
2. Oracle oder echte externe Datenquelle	24
9. Reflexion — Was wurde umgesetzt, was nicht?	24
Umgesetzt ✓	24
Nicht umgesetzt / Abweichungen ✗	25
Fazit	25

1. Projektübersicht

TalentChain ist eine dezentrale Recruiting-Plattform auf der Cardano Blockchain. Sie verbindet reale Recruiting-Ereignisse (Stellenwechsel, Vertragsabschlüsse) mit Smart Contracts, sodass Rewards automatisch und manipulationssicher an alle Mitglieder der Einladungskette ausbezahlt werden.

Kernprinzip: Off-Chain berechnen — On-Chain verifizieren. Talentdaten und Matching-Logik laufen in der PostgreSQL-Datenbank. Die Blockchain übernimmt ausschliesslich: Referral-Beziehungen (unveränderliche UTxOs), Identity NFTs (One-Shot Minting) und Reward-Verteilung (Smart Contract).

1.1 Projektziele

- Manipulationssichere, dauerhafte Referral-Ketten auf der Cardano Blockchain als UTxOs
- Automatische, regelbasierte Reward-Verteilung ohne manuelle Eingriffe
- 1-Personen-1-Account-Prinzip durch Identity NFTs (On-Chain) + Backend-Login (Off-Chain)

- Normalisierte Reward-Verteilung nach aktiven Layern (1–5 Layer, Bottom-Up-Prinzip)
- Modernes TypeScript-Ökosystem mit Lucid Evolution als primäres Off-Chain-Framework

2. Akteure

Akteur	Beschreibung	Rechte
Referral-User	Person mit Cardano Wallet, Teil des Referral-Netzwerks. Registriert via Einladungslink oder als L1 Ambassador durch Admin.	Einladen, Hierarchie anzeigen, Rewards empfangen
Admin	TalentChain-Betreiber. Admin-Wallet-Adresse ist im Smart Contract hardcodiert.	Alle Operationen: L1-Erstellung, Match-Events, User sperren
System / Smart Contract	Aiken-Validator auf Cardano. Prüft Transaktionen automatisch ohne menschlichen Eingriff.	Nur validieren: Transaktionen akzeptieren oder ablehnen
App-Konsument	Nutzt die App ohne Einladungslink. Kein Referrer-Status, keine Rewards.	App nutzen, Link teilen

3. Use Cases

Zusammenfassung — Übersicht aller Use Cases

UC-ID	Name	Status	Bemerkung
UC-01	Wallet verbinden	Implementiert	CIP-30: Eternl, Nami, Lace
UC-02	App-Download teilen	Kein eigener Flow	Technisch möglich, nicht separat umgesetzt
UC-03	Root-User (L1) erstellen	Implementiert	Admin Dashboard: «L1 machen»
UC-04	Einladung annehmen	Implementiert	Kompletter 3-TX-Flow
UC-05	Doppelregistrierung verhindern	Implementiert	Zweifachschutz on-chain + off-chain
UC-06	Eigene Hierarchie anzeigen	Grundversion	Kette sichtbar, kein vollständiger D3-Baum
UC-07	Match-Event auslösen	Implementiert	Admin Dashboard
UC-08	Rewards automatisch verteilen	Teilweise	Validator deployed, aber nicht aktiv im Flow
UC-09	Kette traversieren	Implementiert	On-chain + DB Fallback

UC-10	Reward-Transaktion anzeigen	Implementiert	RewardHistory-Komponente
UC-11	Contract-Laufzeit verwalten	Nicht implementiert	expires_at gesetzt, Renewal-Logik fehlt
UC-12	User sperren (Blacklist)	Implementiert	Kaskadenentfernung aus Kette

UC-01 — Wallet verbinden

Akteur	Referral-User / Admin
Ziel	Cardano Wallet (Eternl, Nami oder Lace) mit dem TalentChain-Frontend verbinden
Vorbedingung	Wallet-Extension im Browser installiert, tADA auf Preprod Testnet vorhanden
Auslöser	User öffnet die TalentChain Web-App und klickt auf «Wallet verbinden»
Hauptablauf	1. App zeigt Wallet-Auswahl (Eternl / Nami / Lace) 2. User wählt Wallet — CIP-30-Dialog erscheint 3. User bestätigt Verbindung 4. App liest getUsedAddresses() bzw. getChangeAddress() 5. Wallet-Adresse und Guthaben werden angezeigt
Ergebnis	Wallet verbunden. Adresse steht für alle weiteren Transaktionen zur Verfügung.
Fehlerfall	Keine Wallet installiert → Fehlermeldung mit Installations-Hinweis
Technisch	WalletConnect.tsx: window.cardano.eternl.enable() via CIP-30
Status	Implementiert

UC-02 — App-Download teilen

Akteur	App-Konsument
Ziel	App-Link teilen ohne Referrer-Status
Hauptablauf	1. User öffnet App ohne Einladungslink 2. App zeigt allgemeinen Download-Link 3. User teilt Link via Social Media oder Messenger
Ergebnis	Link geteilt. Kein Reward — kein Invite-Code, kein NFT.
Hinweis	Kein Invite-Code = keine Referral-Beziehung on-chain. Unterschied zu UC-04.
Status	Technisch möglich, kein separater Flow im Code implementiert

UC-03 — Root-User durch Admin registrieren (L1 Ambassador)

Akteur	Admin
Ziel	Admin erstellt manuell einen L1 Ambassador — erste Ebene im Referral-Netzwerk
Vorbedingung	Admin ist eingeloggt. Ziel-Person hat Wallet und Backend-Account.
Auslöser	Admin klickt «L1 machen» in der User-Verwaltung

Hauptablauf	1. Admin sucht Person im Dashboard 2. Klickt «L1 machen» 3. Backend setzt Rolle auf L1_AMBASSADOR 4. Admin-Wallet mintet Identity-NFT für die Person 5. Referral-UTxO on-chain mit inviter=None (= Root) 6. DB-Eintrag + Invite-Code vergeben
Ergebnis	Person ist L1 Ambassador, kann einladen, erhält Rewards aus der gesamten Downline.
Technisch	api/routes/admin.ts: POST /make-l1/:id · chain/referral-chain.ts: createReferralUtxo(null, pkh)
Status	Implementiert

UC-04 — Einladung annehmen (Referral registrieren)

Akteur	Referral-User
Ziel	Neue Person registriert sich über Einladungslink und speichert die Beziehung on-chain
Vorbedingung	Einlader hat gültigen Identity-NFT und ist in der Referral-Kette. Person hat noch keinen Account.
Auslöser	Person öffnet Registrierungsformular mit Invite-Code des Einladers
Hauptablauf	1. Backend prüft: E-Mail oder Wallet bereits vorhanden? → Ablehnung 2. Backend prüft: Wallet hat bereits Identity-NFT? → Ablehnung 3. Invite-Code prüfen: Einlader aktiv und in der Kette? 4. TX 1: Admin-Wallet mintet Identity-NFT (identity_minting_policy) 5. Warten auf TX-Bestätigung via Blockfrost-Polling (max. 120s) 6. TX 2: Admin-Wallet erstellt Referral-UTxO mit ReferralDatum 7. DB-Eintrag ReferralRelation + Invite-Code vergeben 8. TX 3: Registrierungsbonus CHF 60 ausschütten 9. JWT-Token zurückgeben
Registrierungsbonus	CHF 60 total: 75% (CHF 45) direkter Einlader · 25% (CHF 15) aufgeteilt auf obere Layer
Ergebnis	Person hat Identity-NFT, ist in der Referral-Kette, besitzt eigenen Invite-Code
Fehlerfall 1	Account bereits vorhanden → HTTP 400
Fehlerfall 2	Ungültiger oder gesperrter Invite-Code → HTTP 400
Fehlerfall 3	Wallet hat bereits Identity-NFT → HTTP 400
Technisch	api/routes/auth.ts · chain/mint-nft.ts (TX 1) · chain/referral-chain.ts (TX 2) · chain/registration-bonus.ts (TX 3)
Status	Implementiert

UC-05 — Doppelregistrierung verhindern

Akteur	System (Smart Contract + Backend)
Ziel	Verhindert, dass eine Person mehrere Accounts anlegt
Zweifachschutz 1	Off-Chain: Backend prüft ob E-Mail oder Wallet bereits in PostgreSQL existiert → sofortige Ablehnung
Zweifachschutz 2	On-Chain: hasIdentityNft() prüft via Blockfrost ob Wallet bereits NFT dieser Policy-ID besitzt

Ergebnis	Keine zweite Registrierung möglich — Zweifachschutz on-chain + off-chain
Technisch	chain/mint-nft.ts: hasIdentityNft() · PostgreSQL-Check in api/routes/auth.ts
Status	Implementiert

UC-06 — Eigene Hierarchie anzeigen

Akteur	Referral-User
Ziel	User sieht wer über und unter ihm in der Kette steht
Hauptablauf	1. App liest alle Referral-UTxOs via Blockfrost 2. Rekonstruiert Kette ausgehend vom User 3. Zeigt Einlader oben, eingeladene Personen unten
Datenschutz	Standard: nur Wallet-Adresse sichtbar. Optional: Name-Freigabe aktivierbar.
Ergebnis	Kette mit eigener Position und Layer-Nummer sichtbar.
Technisch	api/routes/referral.ts: GET /my · GET /chain/:id · GET /downline/:id
Status	Grundversion implementiert, kein vollständiger D3.js Baum

UC-07 — Match-Event auslösen und Rewards verteilen

Akteur	Admin
Ziel	Stellenwechsel simulieren und Rewards on-chain an alle aktiven Layer verteilen
Vorbedingung	Mindestens ein Talent ist registriert und in der Referral-Kette. Admin ist eingeloggt.
Auslöser	Admin erstellt Match-Event im Dashboard (Talent, Jahresgehalt, Szenario)
Phase 1: erstellen	1. Admin wählt Talent, Jahresgehalt und Szenario (z.B. CH_LEHRE_WITH_TRAINING = 8%) 2. Gebührenberechnung: Jahresgehalt × Gebührensatz = Gesamtgebühr 3. Aufspaltung: Referrer-Pool (10%), Reg-Pool (5%), Plattform (10%), Rest = Institution + Talent 4. MatchEvent in DB mit Status PENDING gespeichert
Phase 2: verteilen	1. Admin klickt «Rewards verteilen» 2. On-Chain Traversal via traverseChainOnChain() + Blockfrost (Fallback: DB) 3. Blacklistete User überspringen 4. Normalisierte Prozentsätze berechnen 5. Cardano-TX mit mehreren Outputs via Admin-Wallet 6. MatchEvent auf DISTRIBUTED gesetzt, txHash gespeichert
Ergebnis	Rewards in den Wallets aller aktiven Layer (~20s). TX auf preprod.cardanoscan.io sichtbar.
Normalisierung	1 Layer: 100% · 2 Layer: 68.6/31.4% · 5 Layer: 48/22/16/9/5%
Fehlerfall	Keine Referral-Kette → HTTP 400. Event bereits DISTRIBUTED → HTTP 400.
Technisch	api/routes/match.ts · api/routes/rewards.ts · chain/referral-traversal.ts · chain/send-rewards.ts

Status	Implementiert
--------	---------------

UC-08 — Rewards automatisch verteilen (Smart Contract)

Akteur	System (Smart Contract)
Ziel	On-Chain Verifikation dass die Reward-Verteilung korrekt ist
Kernprinzip	Off-chain berechnen → On-chain verifizieren
Hauptablauf	1. reward_distributor empfängt Transaktion 2. Prüft Admin-Signatur 3. Prüft Summe: referrer_sum + institution + talent + platform == total_fee 4. Prüft mindestens ein Output vorhanden 5. Alle Checks OK → Transaktion akzeptiert
Fehlerfall	Summe stimmt nicht → Contract lehnt ab. Keine tADA werden bewegt.
Technisch	reward_distributor.ak — Aiken Validator deployed auf Preprod
Status	Deployed aber nicht aktiv im Flow — Rewards gehen direkt vom Admin-Wallet via CSL

Hinweis: Der reward_distributor Validator ist korrekt implementiert und auf Preprod deployed. Er wird aber aktuell nicht im Reward-Flow aufgerufen. Die Rewards werden direkt vom Admin-Wallet via cardano-tx.ts gesendet. Full Smart-Contract-enforced Distribution ist der nächste Schritt.

UC-09 — Kette traversieren (Off-Chain)

Akteur	System (Backend TypeScript)
Ziel	Vollständige Referral-Kette lesen und normalisierte Reward-Beträge berechnen
Hauptablauf	1. Alle Referral-UTxOs von Script-Adresse laden 2. UTxO des Talents finden (invitee == talent_pkh) 3. inviter-Feld lesen → nächsten UTxO suchen 4. Wiederholen bis inviter=None oder 5 Layer erreicht 5. Blacklistete User überspringen 6. Normalisierte Prozentsätze berechnen 7. Mit Referrer-Pool multiplizieren
Ergebnis	Liste mit Wallet-Adresse + CHF-Betrag pro aktivem Layer
Technisch	chain/referral-chain.ts: traverseChainOnChain() · chain/referral-traversal.ts: calculateRewards()
Status	Implementiert — On-chain primär, DB als Fallback

UC-10 — Reward-Transaktion anzeigen

Akteur	Referral-User
Ziel	User sieht seine eigene Reward-History aus der Blockchain

Hauptablauf	1. App lädt TX-History der Wallet via Blockfrost 2. Gleicht TXs mit DISTRIBUTED Match-Events ab 3. Zeigt TalentChain-Rewards separat von anderen Transaktionen
Ergebnis	Liste aller erhaltenen Rewards mit TX-Hash, Betrag, Datum und Link zu Cardanoscan.
Technisch	api/routes/rewards.ts: GET /history · Frontend: _rewards.tsx RewardHistory-Komponente
Status	Implementiert

UC-11 — Contract-Laufzeit verwalten

Akteur	Admin / System
Ziel	Ablaufdatum einer Referral-Beziehung verlängern
Hauptablauf	1. System prüft expires_at-Feld im ReferralDatum 2. Admin löst Renew-Transaktion aus 3. referral_registry Validator prüft Admin-Signatur 4. UTxO wird mit neuem expires_at Datum aktualisiert
Ergebnis	Referral-Beziehung verlängert, neues expires_at on-chain gespeichert.
Technisch	referral_registry.ak: Renew-Redeemer mit Admin-Signatur · expires_at ist im ReferralDatum gesetzt
Status	expires_at im Datum gesetzt, Renewal-Logik noch nicht implementiert

UC-12 — User sperren (Blacklist)

Akteur	Admin
Ziel	User sperren und automatisch aus der Referral-Kette entfernen
Vorbedingung	Admin ist eingeloggt, User existiert
Hauptablauf	1. Admin klickt «Sperren» in der User-Verwaltung 2. User wird in DB auf isBlacklisted = true gesetzt 3. cascadeRemoveFromChain(): eigene ReferralRelation gelöscht, Kinder zum Parent verknüpft 4. Invite-Code des Users wird auf null gesetzt
Ergebnis	User gesperrt. Kinder in der Kette aktiv (verbunden mit nächsthöherem Layer). On-Chain UTxOs bleiben bestehen.
Hinweis	Blockchain-UTxOs können nicht gelöscht werden. Die Off-Chain-DB dient als operativer Filter.
Technisch	api/routes/admin.ts: POST /blacklist/:id · chain/chain-utils.ts: cascadeRemoveFromChain()
Status	Implementiert

4. Technologie-Stack

4.1 Smart Contracts — Aiken

Aiken ist die Sprache für alle On-Chain Validators und kompiliert direkt zu Plutus V3 — dem nativen Format der Cardano Blockchain.

Warum Aiken? Es ist die einzige Sprache die speziell für Cardano gebaut wurde, hat ein eingebautes Testframework und braucht nur ein einziges CLI-Tool.

Validator	Funktion	Status im Code
identity_minting_policy	Nur Admin darf NFTs minen	Aktiv genutzt
referral_registry	Speichert Referral-Beziehungen als UTxO	Aktiv genutzt
reward_distributor	Verifiziert Reward-Beträge	Deployed, aber aktuell nicht im Flow

Wichtig: Aiken ist ausschliesslich für On-Chain-Validierung zuständig. Transaktionen werden immer off-chain in TypeScript gebaut und nur on-chain verifiziert.

4.2 Off-Chain SDK — Lucid Evolution

Ursprünglich war das Evolution SDK geplant. Gewechselt zu Lucid Evolution weil es battle-tested ist, eine grosse Community hat und Plutus V3 nativ unterstützt. Das Evolution SDK war zum Zeitpunkt der Implementierung noch nicht stabil genug.

- Battle-tested mit vielen produktiven Projekten auf Cardano
- Dokumentation für komplexe Operationen (Datum-Encoding, overrideUTxOs) deutlich umfassender
- Nativer Plutus V3 Support — kompatibel mit den Aiken-kompilierten Contracts

Einsatz: NFT Minting, Referral-UTxOs erstellen, Script-Adressen berechnen.

4.3 Cardano Serialization Library (CSL)

Für alle direkten Zahlungen (Rewards, Registrierungsbonus). Lucid baut die Transaktion, CSL signiert mit dem Admin-Key aus der .env und submittet via Blockfrost.

- Direkte Kontrolle über TX-Bau, Inputs und Outputs

- Für reine Zahlungen ist der volle Lucid-Stack nicht nötig
- Klare Aufgabenteilung: Lucid baut — CSL signiert und submittet

4.4 Blockfrost

Ersetzt einen eigenen Cardano-Node komplett. Free Tier reicht für Entwicklung und Demo aus.

- Kein eigener Node nötig — massiv tiefere Infrastrukturkosten
- Nr. 1 im Cardano-Ökosystem (Developer Survey 2025: 24.6%)
- Einsatz: UTxO-Abfragen, TX-Submit, TX-Bestätigung polling, Datum-Lookup, TX-History

4.5 Backend — Node.js + TypeScript

Express REST API mit zwei Middleware-Ebenen: JWT-Check für alle eingeloggten User, zusätzlicher Admin-Check für sensitive Operationen.

- Selbe Sprache (TypeScript) für Backend und Frontend — kein Kontextwechsel
- TypeScript end-to-end verhindert Fehler zwischen DB, Blockchain und API
- Middleware-Trennung: authMiddleware (JWT) + adminMiddleware (Rollencheck)

4.6 Datenbank — PostgreSQL + Prisma

Speichert User, Referral-Beziehungen und Match-Events. Die DB spiegelt die Blockchain-Kette wider ist aber nicht autoritativ — bei der Reward-Verteilung wird zuerst on-chain traversiert, die DB dient als Fallback.

- Relationale Daten passen natürlich in ein SQL-Schema
- Prisma generiert typesichere Queries — keine SQL-Injection-Risiken

4.7 Authentifizierung — JWT + bcrypt

JWT Tokens mit 7 Tagen Laufzeit. Passwörter mit bcrypt gehasht.

- Stateless Auth — kein Session-Management nötig
- Zweifachschutz: On-Chain Identity-NFT + Off-Chain Login verhindert Doppelregistrierung

4.8 Frontend — Next.js 14 + React

Next.js App Router mit CSS-Variablen für Dark/Light Theme und Framer Motion für Animationen.

- Industriestandard für React-Applikationen
- CSS-Variablen für konsistentes Theming ohne externe Bibliothek

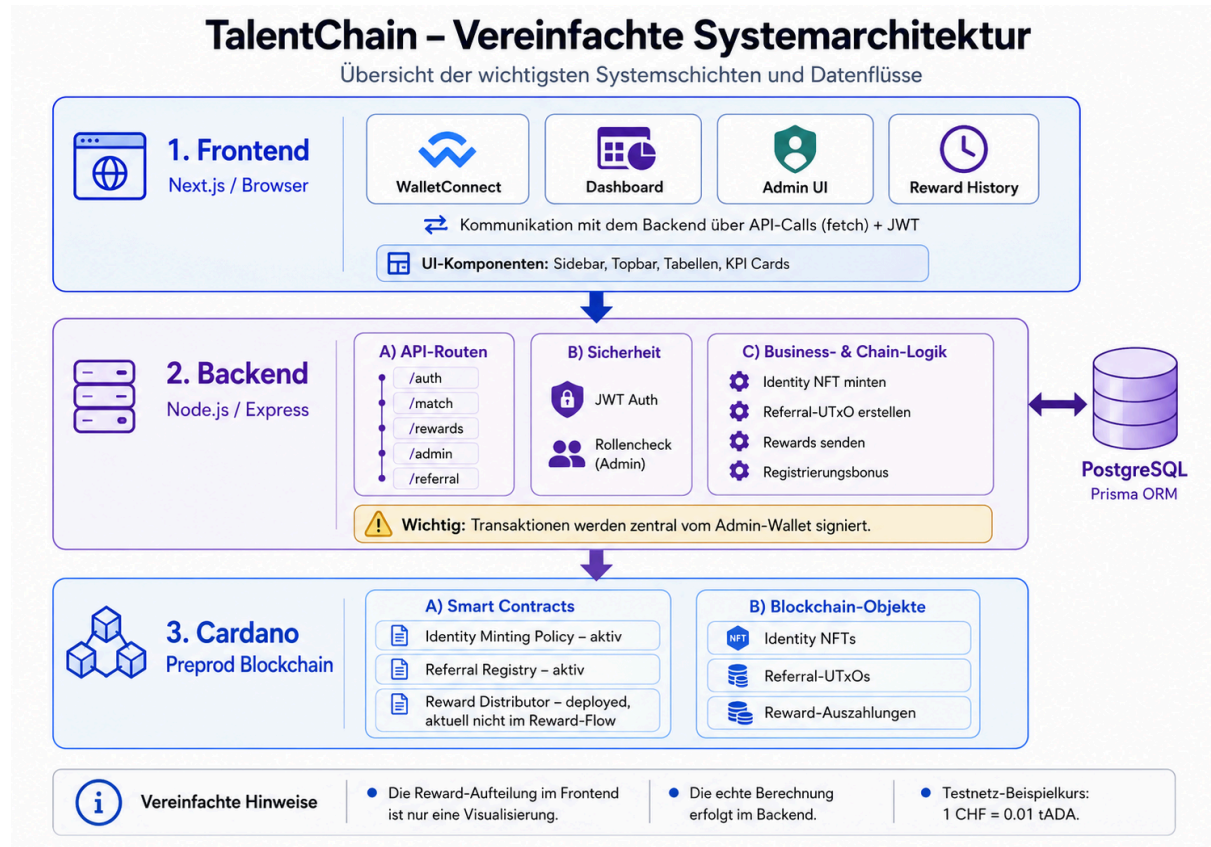
Wichtig: Alle Blockchain-Transaktionen laufen über das Backend. Der User signiert nie selbst — das macht immer der Admin via Seed Phrase in der .env.

4.9 Zusammenfassung

Schicht	Technologie	Warum
Smart Contracts	Aiken → Plutus V3	Einzige produktionsreife Cardano-Sprache
Off-Chain SDK	Lucid Evolution	Battle-tested, Plutus V3, grosse Community
TX-Signing	CSL (Emurgo)	Direkte Kontrolle für Zahlungen
Provider	Blockfrost	Kein eigener Node nötig
Backend	Node.js + TypeScript	Selbe Sprache wie Frontend
Datenbank	PostgreSQL + Prisma	Typsichere Queries, relationale Daten
Auth	JWT + bcrypt	Stateless, sicher
Frontend	Next.js 14 + React	Industriestandard
Wallet	CIP-30 (Eternl/Nami/Lace)	Cardano Browser-Standard
Testnet	Cardano Preprod	Offizielle Testumgebung

5. Systemarchitektur

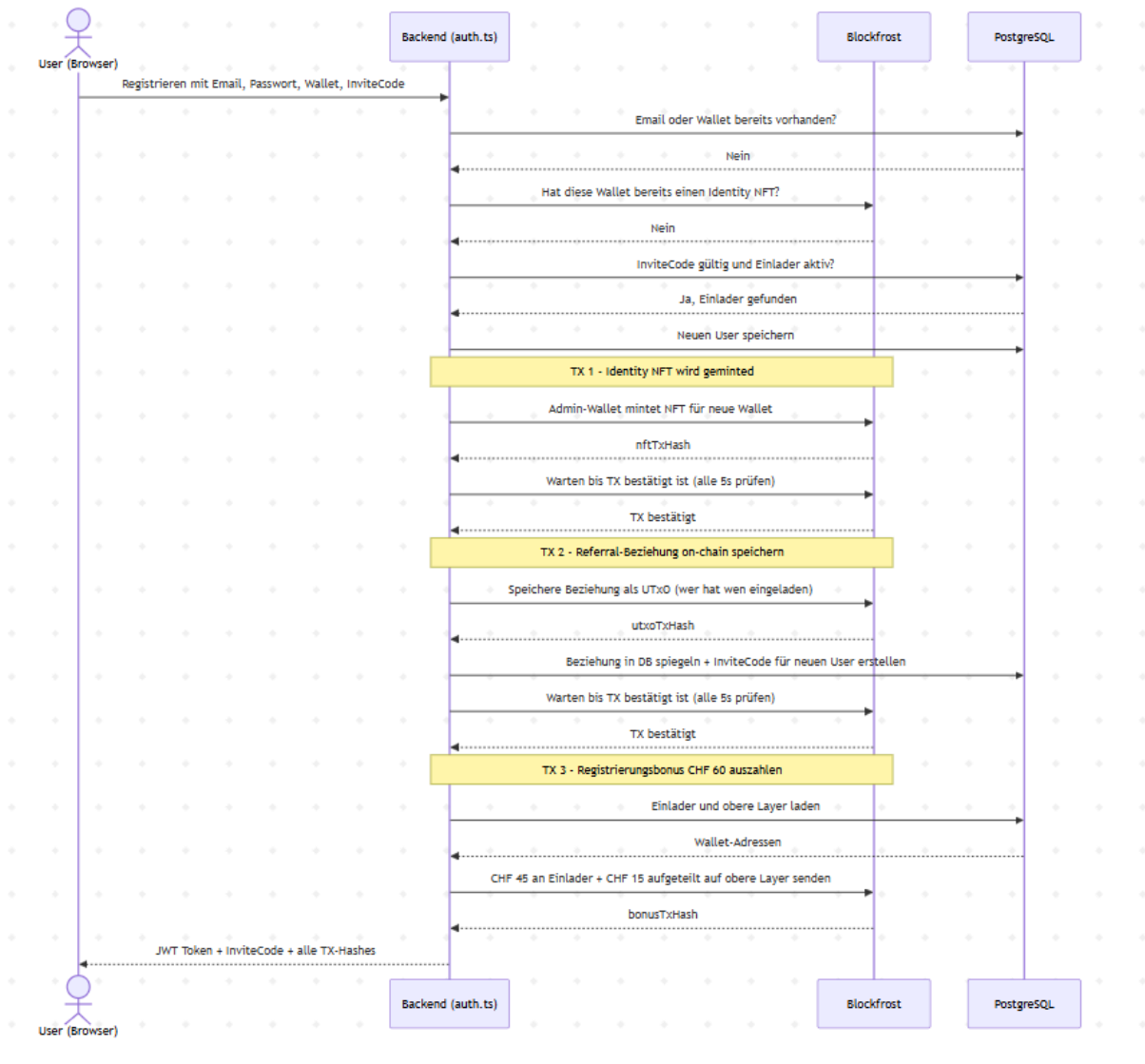
TalentChain ist in drei klar getrennte Schichten aufgeteilt:



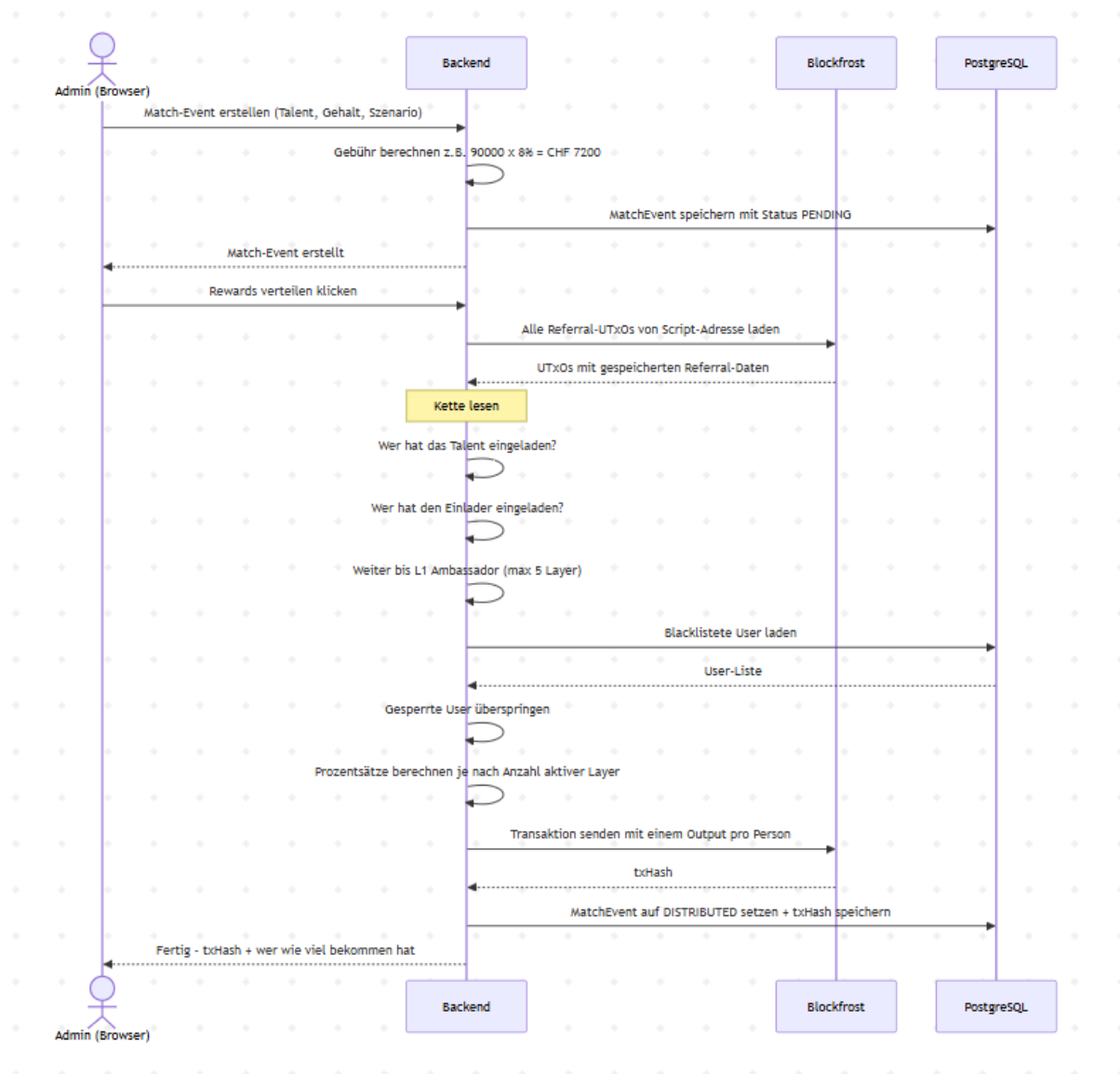
6. Sequenzdiagramme

6.1 Registrierung mit Einladungslink (3-TX-Flow)

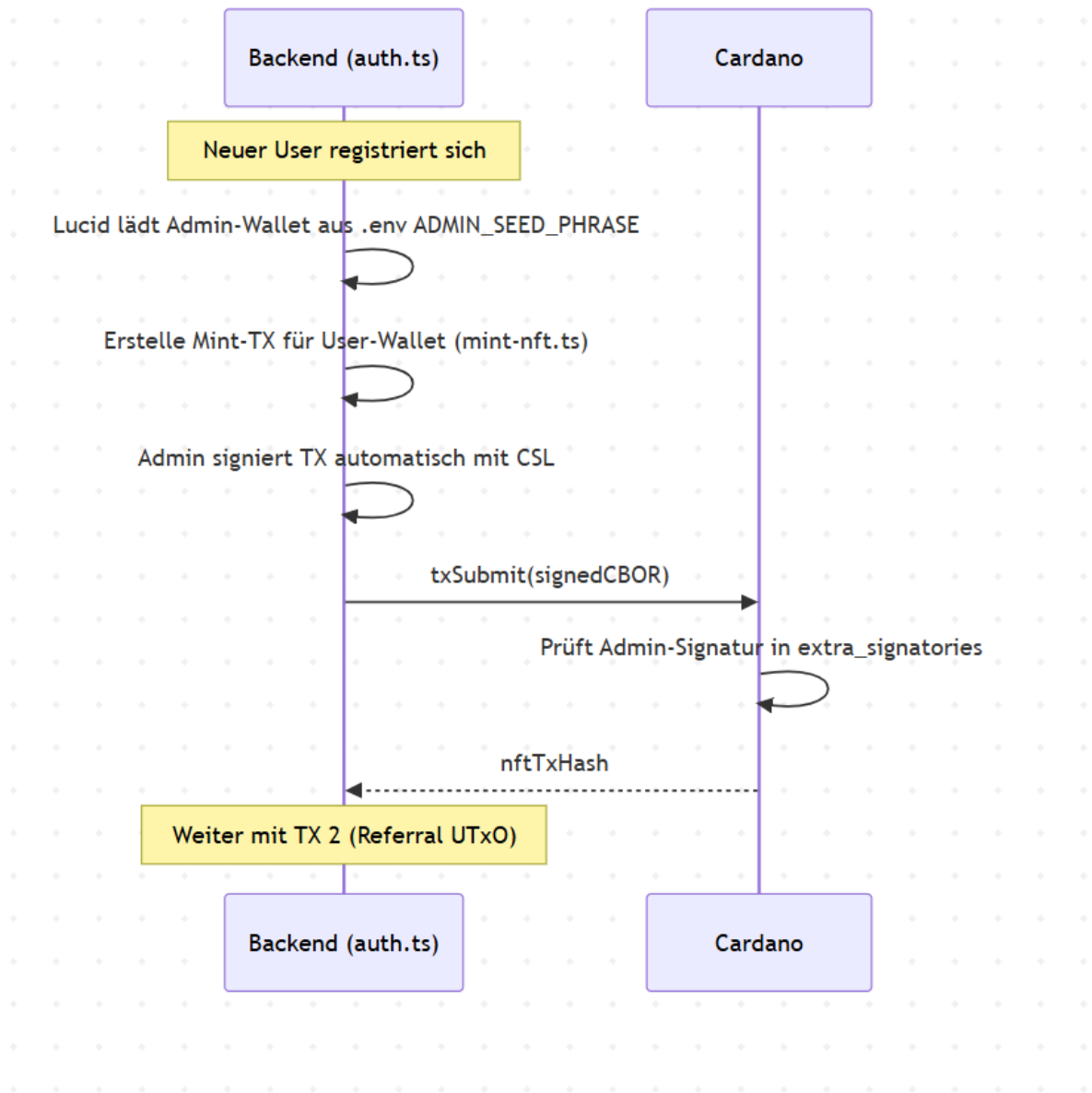
Dieser Flow zeigt den komplexesten Blockchain-Teil: drei sequenzielle, voneinander abhängige Cardano-Transaktionen.



6.2 Reward-Verteilung (Match-Event)



6.3 Identity NFT Minting — Aiken Validator Interaktion



7. Testkonzept

7.1 Warum Testing bei Smart Contracts besonders wichtig ist

Smart Contracts auf der Blockchain sind nach dem Deployment unveränderlich. Ein Fehler in einem Validator kann nicht gepatcht werden — er ist permanent. Zudem arbeiten Contracts direkt mit Geldwerten.

Kritischer Punkt: Ein fehlerhafter Summencheck im reward_distributor könnte bedeuten, dass Transaktionen mit falschen Beträgen akzeptiert werden. Der Schaden wäre irreversibel und direkt finanziell. Deshalb gilt: testen vor dem Deploy, und nach jeder Änderung am Flow manuell verifizieren.

7.2 Eingesetzte Test-Methoden

Methode	Tool	Scope
API-Tests	Postman	Alle REST-Endpunkte: Auth, Match, Rewards, Admin, Referral
Manuelle End-to-End Tests	Browser + Dashboard	Komplette Applikation inkl. Blockchain-Flows
Negativtests	Postman + Browser	Ungültige Eingaben, fehlende Berechtigungen, Fehlerfälle
On-Chain Verifikation	preprod.cardanoscan.io	TX-Hashes nach jedem Blockchain-Flow prüfen
Skript-Tests	ts-node	NFT-Status-Check, Admin-Seed, Invite-Code Migration

7.3 API-Tests (Postman) — Happy Path

Alle Routes wurden mit Postman getestet. Dieser Abschnitt zeigt die Positivfälle — das erwartete Verhalten bei gültigen Eingaben.

Auth-Routes /api/auth

Testfall	Erwartetes Ergebnis	Status
POST /register mit gültigem Invite-Code	HTTP 200, JWT Token, nftTxHash vorhanden	OK
POST /register ohne Invite-Code	HTTP 200, kein NFT, nicht in Kette	OK
POST /login mit korrekten Daten	HTTP 200, JWT Token zurück	OK

Match-Routes /api/match

Testfall	Erwartetes Ergebnis	Status
POST /create als Admin	HTTP 200, MatchEvent mit Status PENDING	OK
GET /events als Admin	HTTP 200, alle Events zurück	OK
GET /events als User	HTTP 200, nur eigene Events	OK
POST /cancel/:id auf PENDING Event	HTTP 200, Status CANCELLED	OK

Reward-Routes /api/rewards

Testfall	Erwartetes Ergebnis	Status
POST /distribute mit gültigem Match-Event	HTTP 200, txHash vorhanden	OK
GET /history mit gültigem Token	HTTP 200, TX-Liste der Wallet	OK

Admin-Routes /api/admin

Testfall	Erwartetes Ergebnis	Status
GET /users als Admin	HTTP 200, alle User	OK
POST /make-l1/:id	HTTP 200, Rolle L1_AMBASSADOR, Invite-Code vergeben	OK
POST /blacklist/:id	HTTP 200, isBlacklisted true, Kaskade ausgeführt	OK
POST /unblacklist/:id	HTTP 200, isBlacklisted false	OK
GET /referral-script-address	HTTP 200, Preprod Script-Adresse zurück	OK

7.4 Negativtests — Fehlerfälle & Sicherheit

Negativtests prüfen ob das System ungültige Eingaben, fehlende Berechtigungen und Missbrauchsversuche korrekt ablehnt. Ein System das nur im Happy Path funktioniert ist nicht produktionsreif.

Authentifizierung & Zugriffskontrolle

Testfall	Erwartetes Verhalten	Status
----------	----------------------	--------

API-Aufruf ohne JWT Token	HTTP 401, Kein Token	Korrekt abgelehnt
API-Aufruf mit abgelaufenem Token	HTTP 401, Ungültiger Token	Korrekt abgelehnt
API-Aufruf mit manipuliertem Token	HTTP 401, Ungültiger Token	Korrekt abgelehnt
Admin-Route als normaler User aufrufen	HTTP 403, Keine Admin-Berechtigung	Korrekt abgelehnt
Login mit falschem Passwort	HTTP 401, Ungültige Anmeldedaten	Korrekt abgelehnt
Login mit gesperrtem Account	HTTP 403, Account gesperrt	Korrekt abgelehnt
GET /users als normaler User	HTTP 403, Keine Admin-Berechtigung	Korrekt abgelehnt

Registrierung & Doppelregistrierung

Testfall	Erwartetes Verhalten	Status
Registrierung mit bereits verwendeter E-Mail	HTTP 400, Account existiert bereits	Korrekt abgelehnt
Registrierung mit bereits verwendeter Wallet	HTTP 400, Account existiert bereits	Korrekt abgelehnt
Registrierung mit ungültigem Invite-Code	HTTP 400, Ungültiger Einladungscode	Korrekt abgelehnt
Registrierung mit Invite-Code eines gesperrten Users	HTTP 400, Einladungscode nicht mehr gültig	Korrekt abgelehnt
Registrierung: Wallet hat bereits Identity NFT	HTTP 400, Wallet hat bereits einen Identity NFT	Korrekt abgelehnt
Registrierung ohne Pflichtfelder (E-Mail fehlt)	HTTP 400, Alle Felder erforderlich	Korrekt abgelehnt
Registrierung mit Invite-Code eines Users der nicht in der Kette ist	HTTP 400, Einladungscode nicht mehr gültig	Korrekt abgelehnt

Match-Events & Rewards

Testfall	Erwartetes Verhalten	Status
Match-Event erstellen als normaler User	HTTP 403, Nur Admins können Match-Events erstellen	Korrekt abgelehnt
Match-Event ohne Pflichtfelder erstellen	HTTP 400, Fehlermeldung	Korrekt abgelehnt
Rewards verteilen: Event bereits DISTRIBUTED	HTTP 400, Rewards bereits verteilt	Korrekt abgelehnt
Rewards verteilen: Event ist CANCELLED	HTTP 400, Event wurde storniert	Korrekt abgelehnt

Rewards verteilen: kein Talent in der Kette	HTTP 400, Keine Referral-Kette gefunden	Korrekt abgelehnt
Event stornieren das bereits DISTRIBUTED ist	HTTP 400, Nur PENDING Events können storniert werden	Korrekt abgelehnt
Event stornieren als normaler User	HTTP 403, Nur Admins können Events stornieren	Korrekt abgelehnt
Reward History ohne Token abrufen	HTTP 401, Kein Token	Korrekt abgelehnt

Blacklist & Referral-Kette

Testfall	Erwartetes Verhalten	Status
Gesperrter User versucht einzuladen	HTTP 400, Einladungscode nicht mehr gültig	Korrekt abgelehnt
Rewards an gesperrten User in der Kette	Gesperrter User wird übersprungen, kein tADA erhalten	Korrekt abgelehnt
Nicht-existierenden User sperren	HTTP 404, User nicht gefunden	Korrekt abgelehnt
Nicht-existierendes Match-Event verteilen	HTTP 404, Match-Event nicht gefunden	Korrekt abgelehnt
PKH aus ungültiger Wallet-Adresse ableiten	HTTP 400, Ungültige Adresse	Korrekt abgelehnt

On-Chain Sicherheit

Testfall	Erwartetes Verhalten	Status
NFT Mint ohne Admin-Signatur (identity_minting_policy)	Validator lehnt TX ab — kein Mint möglich	Korrekt abgelehnt
Referral UTxO mit nur einer Signatur (referral_registry)	Validator lehnt TX ab — UTxO nicht erstellt	Korrekt abgelehnt
reward_distributor: Summe stimmt nicht (referrer+institution+talent+platform $\neq$ total_fee)	Validator lehnt TX ab — keine tADA bewegt	Korrekt abgelehnt
Zweite Wallet-Registrierung mit derselben Wallet (NFT bereits vorhanden)	hasIdentityNft() erkennt NFT → HTTP 400	Korrekt abgelehnt

7.5 Manuelle End-to-End Tests

Die gesamte Applikation wurde manuell im Browser getestet. Jeder Flow wurde komplett von Anfang bis Ende durchgespielt und das Ergebnis on-chain auf preprod.cardanoscan.io verifiziert.

Registrierung & Wallet

Testfall	Ergebnis
Wallet verbinden mit Eternl (CIP-30)	Adresse korrekt ausgelesen
Registrierung mit Invite-Code — 3-TX-Flow komplett	NFT auf Wallet, UTxO on-chain, Bonus ausbezahlt
Registrierung ohne Invite-Code	Account erstellt, kein NFT, nicht in Kette
Doppelregistrierung: gleiche E-Mail	Abgelehnt mit Fehlermeldung
Doppelregistrierung: gleiche Wallet	Abgelehnt via NFT-Check on-chain
Login mit korrekten Daten	JWT gesetzt, Dashboard geladen
Login mit falschem Passwort	Fehlermeldung angezeigt

Admin Dashboard

Testfall	Ergebnis
User zu L1 Ambassador machen	Rolle gesetzt, UTxO mit inviter=None on-chain
Match-Event erstellen (Talent, Gehalt, Szenario)	Event mit Status PENDING in Tabelle
Rewards verteilen — on-chain Traversal	TX auf Cardanoscan sichtbar, Wallets erhalten tADA
Rewards verteilen — mit gesperrtem User in Kette	Gesperrter User übersprungen, Rest erhält anteilig mehr
Event stornieren	Status CANCELLED, keine Verteilung mehr möglich
User sperren	isBlacklisted=true, Kinder zum Parent verknüpft
User entsperren	isBlacklisted=false, Kette muss manuell neu gesetzt werden
Einlader manuell setzen	Referral-Beziehung in DB erstellt
User aus Kette entfernen	Kaskade ausgeführt, Kinder neu verknüpft

Referral-Kette & Reward-Verteilung

Testfall	Ergebnis
Eigene Position in der Kette anzeigen	Layer-Nummer korrekt, Einlader sichtbar
Reward History anzeigen	TalentChain-TXs separat markiert, Link zu Cardanoscan
Normalisierung bei 1 aktivem Layer	100% an einzigen Referrer
Normalisierung bei 2 aktiven Layern	68.6% / 31.4% korrekt aufgeteilt

Normalisierung bei gesperrtem Layer	Prozentsätze auf verbleibende Layer angepasst
Registrierungsbonus — Aufteilung 75% / 25%	CHF 45 direkter Einlader, CHF 15 obere Layer

On-Chain Verifikation (Cardanoscan)

Was geprüft	Ergebnis
NFT Mint TX — Asset auf Empfänger-Wallet sichtbar	Korrekt
Referral UTxO — Inline Datum an Script-Adresse vorhanden	Korrekt
Reward TX — Mehrere Outputs an korrekte Adressen	Korrekt
Registrierungsbonus TX — Beträge stimmen mit 75/25-Aufteilung	Korrekt
L1 Root UTxO — inviter=None im Datum	Korrekt

7.6 Bewusst nicht getestete Bereiche

Bereich	Grund
Automatisierte E2E-Tests (Playwright / Cypress)	Aufwand unverhältnismässig für Demo-Phase; Framer-Motion-Animationen und komplexe Dashboard-UI erschweren Automatisierung
On-Chain Traversal unter Last (viele UTxOs)	Preprod hat natürliche Limitierungen; Performance-Tests nicht zielführend
Oracle-Integration	Modul nicht implementiert — Admin-Trigger ersetzt Oracle manuell
Laufzeit-Management / Renewal (UC-11)	expires_at-Feld gesetzt, Renewal-Logik noch nicht implementiert
Dual Signing (Einlader + Invitee)	Für Demo-Phase zurückgestellt; Admin signiert stellvertretend

8. Reflexion — Feedback aus der Zwischenpräsentation

Aufgenommenes Feedback

1. SDK-Wahl: Evolution SDK zu Lucid Evolution

In der Zwischenpräsentation wurde empfohlen, das ursprünglich geplante Evolution SDK nicht weiterzuverwenden, da es zum Zeitpunkt der Umsetzung noch nicht stabil genug für

unseren Use Case war. Dieses Feedback wurde vollständig aufgenommen und die Off-Chain-Blockchain-Logik auf Lucid Evolution umgestellt.

Lucid Evolution ist im Cardano-Ökosystem etablierter, besser dokumentiert und unterstützt Plutus V3 nativ — was direkt zu unseren Aiken-kompilierten Smart Contracts passt. Alle zentralen Blockchain-Interaktionen (NFT Minting, Referral-UTxOs, Script-Adressen) laufen seither über Lucid Evolution.

2. Off-Chain berechnen, On-Chain verifizieren

Die Empfehlung, die Blockchain nicht für unnötige Berechnungen zu verwenden, wurde als zentrales Architekturprinzip übernommen. Reward-Berechnungen und Referral-Traversal laufen vollständig im TypeScript-Backend — der Smart Contract dient ausschliesslich als Vertrauensschicht und prüft, ob die gesendeten Beträge den vereinbarten Regeln entsprechen.

Dieses Prinzip macht das System effizienter, günstiger im Betrieb und deutlich einfacher testbar, da die Berechnungslogik in Standard-TypeScript-Tests abgedeckt werden kann, ohne jedes Mal eine Blockchain-Transaktion auszulösen.

3. Kein eigener Cardano-Node in der Demo-Phase

Aus der technischen Analyse und dem Feedback ergab sich, dass für die Demo-Phase kein eigener Cardano-Node erforderlich ist. Blockfrost übernimmt alle notwendigen Aufgaben: UTxO-Abfragen, Transaktionsbestätigungen, Datum-Lookups, TX-History und das Einreichen von Transaktionen. Die Infrastruktur bleibt dadurch deutlich einfacher, günstiger und für das Preprod Testnet vollständig ausreichend.

Nicht aufgenommenes oder bewusst zurückgestelltes Feedback

1. Dual Signing durch Einlader und neue Person

Die Empfehlung, dass Einlader und neue Person die Referral-Transaktion jeweils mit der eigenen Wallet selbst signieren, wurde für die Demo-Phase bewusst zurückgestellt. Aktuell signiert das Admin-Backend stellvertretend für beide Parteien, was den Ablauf für Demo-Zwecke stark vereinfacht.

Für eine produktive Version wäre Dual Signing der korrektere Ansatz: Beide Parteien würden ihre Beziehung eigenständig on-chain bestätigen. Der referral_registry Validator ist fachlich bereits auf dieses Modell ausgerichtet; der vollständige Frontend-Flow wurde jedoch noch nicht implementiert.

2. Oracle oder echte externe Datenquelle

Das Feedback wies darauf hin, dass das System langfristig eine verifizierbare Verbindung zur echten Welt benötigt. In der Demo wird dieses Ereignis manuell durch den Admin ausgelöst. Ein Oracle-Modul wurde für die Demo-Phase bewusst nicht umgesetzt, da der Aufwand im Verhältnis zum Demonstrationswert zu hoch gewesen wäre.

Für eine produktive Version müsste geprüft werden, wie reale Recruiting-Ereignisse verlässlich bestätigt werden können — etwa durch signierte Unternehmensbestätigungen, interne Freigaben oder ausgewählte Validatoren.

Persönliches Feedback — Silvan

Feedback aus dem persönlichen Gespräch mit Silvan (Cardano-Experte):

Feedback von Silvan

[Hier persönliches Feedback eintragen]

Persönliches Feedback — Flavio

Feedback aus dem persönlichen Gespräch mit Flavio (Coach):

Feedback von Flavio

[Hier persönliches Feedback eintragen]

9. Reflexion — Was wurde umgesetzt, was nicht?

Umgesetzt

Der gesamte Kernflow — Identity NFT Minting, On-Chain Referral-Beziehungen und Reward-Verteilung — ist auf dem Cardano Preprod Testnet funktionsfähig implementiert und manuell getestet. Alle drei Aiken-Validatoren sind deployed; der Identity Minting Validator und der Referral Registry Validator werden aktiv im Flow verwendet.

Die Dokumentation spiegelt den tatsächlich gebauten Prototyp vollständig wider. Abweichungen zwischen Konzept und Umsetzung sind explizit ausgewiesen (UC-08 reward_distributor, UC-11 Laufzeit-Management, D3-Baum) und sachlich begründet. Es wurden keine Features im Nachhinein in die Dokumentation aufgenommen, die nicht tatsächlich implementiert wurden.

Komponente	Status	Bemerkung
Aiken Smart Contracts (alle 3)	Deployed	identity_minting_policy, referral_registry, reward_distributor auf Preprod
Identity NFT Minting	Funktionsfähig	Via Lucid Evolution + Aiken Validator (Admin-Signatur-Check)
Referral-UTxO on-chain	Funktionsfähig	Inline Datum mit vollständiger ReferralDatum Struktur
On-Chain Chain-Traversal	Funktionsfähig	Via Blockfrost UTxO-Abfragen + CBOR Datum-Dekodierung
Reward-Verteilung	Funktionsfähig	Normalisierte Prozentsätze, Multi-Output TX via CSL
Registrierungsbonus (CHF 60)	Funktionsfähig	3-TX-Flow mit TX-Bestätigungs-Polling (max. 120s)
Blacklist / Kaskadenentfernung	Funktionsfähig	DB-seitig, on-chain UTxOs bleiben (unveränderlich)
JWT-Authentifizierung	Funktionsfähig	Admin + User Rollen, bcrypt Password-Hashing
Admin Dashboard	Funktionsfähig	User, Events, Rewards, Referral-Kette, alle CRUD-Operationen
Wallet-Verbindung (Frontend)	Funktionsfähig	CIP-30: Eternl / Nami / Lace

Nicht umgesetzt / Abweichungen

Die dokumentierten Abweichungen entsprechen bewussten Entscheidungen im Projektverlauf und sind keine ungeplanten Ausfälle. Sie sind vollständig und transparent in der Dokumentation ausgewiesen.

Komponente	Status	Grund
Oracle-Modul	Nicht implementiert	Zu komplex für Demo-Phase. Admin-Trigger ersetzt Oracle manuell.
Dual-Signing (Einlader + Invitee)	Nicht implementiert	Admin signiert stellvertretend. Vereinfacht UX für Demo massiv.
reward_distributor als Spend-Validator	Teilweise	Validator deployed und verifiziert Summen. Aktuelle Distribution bypassed Spend-Flow — direkte CSL-Zahlung.
Evolution SDK	Ersetzt	Evolution SDK war zum Zeitpunkt nicht produktionsreif. Ersetzt durch Lucid Evolution.
D3.js Referral-Baum	Grundversion	Grundlegende Ketten-Anzeige vorhanden, kein vollständiger interaktiver D3-Baum.

Laufzeit-Management (UC-11)	Nicht implementiert	expires_at-Feld im Datum gesetzt, Renewal-Logik noch nicht aktiv.
-----------------------------	---------------------	---

Fazit

Der Kernflow von TalentChain — Identity NFT Minting, On-Chain Referral-Beziehungen und Reward-Verteilung — ist auf dem Cardano Preprod Testnet vollständig funktionsfähig und getestet. Alle drei Aiken-Validatoren sind deployed. Das zentrale Architekturprinzip, Blockchain ausschliesslich für Vertrauen einzusetzen und nicht für alles, wurde konsequent umgesetzt: Talentdaten und Matching-Logik bleiben off-chain, die Blockchain-Schicht ist minimal und auditierbar.

Die dokumentierten Abweichungen (reward_distributor als aktiver Spend-Validator, Dual Signing, Oracle, D3-Baum, Laufzeit-Management) sind bewusste Zurückstellungen für eine produktive Weiterentwicklung — nicht ungeplante Ausfälle. Zwischen der vorliegenden Dokumentation und dem tatsächlich gebauten Prototyp besteht vollständige Konsistenz.